

```
#Importing Important Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import keras
from keras.models import Sequential
from keras.layers import Dense, Conv2D, MaxPool2D, Flatten, Dropout, BatchNormalization
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.model_selection import train_test_split
import cv2
from sklearn.metrics import classification_report, confusion_matrix
from keras.callbacks import ReduceLROnPlateau
import os
```

```
# Pneumonia dataset
labels= ['PNEUMONIA','NORMAL']
img_resize = 150

def get_data(dir):
    data= []
    for label in labels:
        path = os.path.join(dir,label)
        class_num = labels.index(label)

        for img in os.listdir(path):
            try:
                if img.endswith('jpeg'):
                    img_arr = cv2.imread(os.path.join(path,img),cv2.IMREAD_GRAYSCALE)
                    if img_arr is not None:
                        resized_arr = cv2.resize(img_arr,(img_resize,img_resize))
                        data.append([resized_arr, class_num])
            except Exception as e:
                print(e)
    return np.array(data, dtype='object')
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import os
import shutil
import random

# paths
train_dir = "/content/drive/MyDrive/Pneumonia detection/chest_xray/train"
val_dir = "chest_xray/val"

classes = ["NORMAL", "PNEUMONIA"]
split_ratio = 0.2 # 20% for validation

# create val directories
for cls in classes:
    os.makedirs(os.path.join(val_dir, cls), exist_ok=True)

# split data
for cls in classes:
    cls_path = os.path.join(train_dir, cls)
    images = os.listdir(cls_path)
    random.shuffle(images)

    val_size = int(len(images) * split_ratio)
    val_images = images[:val_size]

    for img in val_images:
        src = os.path.join(cls_path, img)
        dst = os.path.join(val_dir, cls, img)
        shutil.move(src, dst)

print("Validation set created successfully!")
```

Validation set created successfully!

```
train = get_data('/content/drive/MyDrive/Pneumonia detection/chest_xray/train')
test = get_data('/content/drive/MyDrive/Pneumonia detection/chest_xray/test')
val = get_data('/content/drive/MyDrive/Pneumonia detection/chest_xray/val')
```

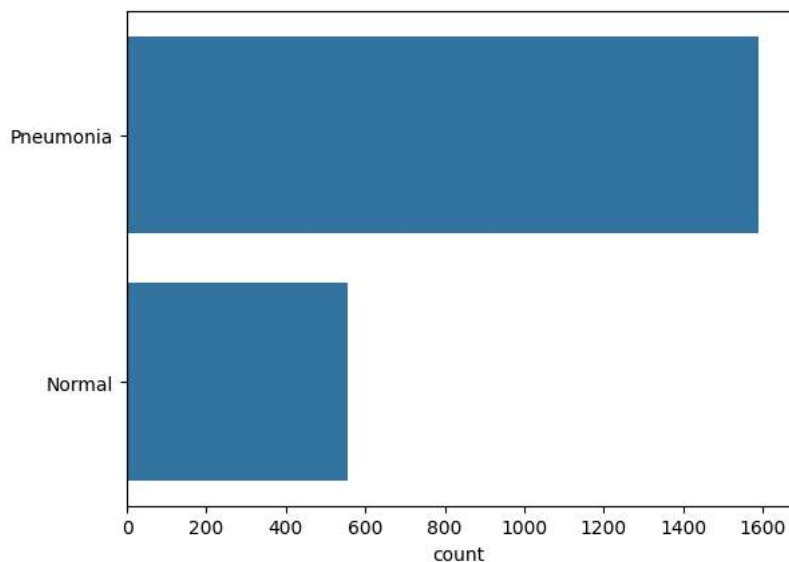
Start coding or [generate](#) with AI.

```
#data visualization and preprocessing
```

```
l = []
for i in train:
    if(i[1] == 0):
        l.append('Pneumonia')
    else:
        l.append('Normal')
```

```
sns.countplot(l)
```

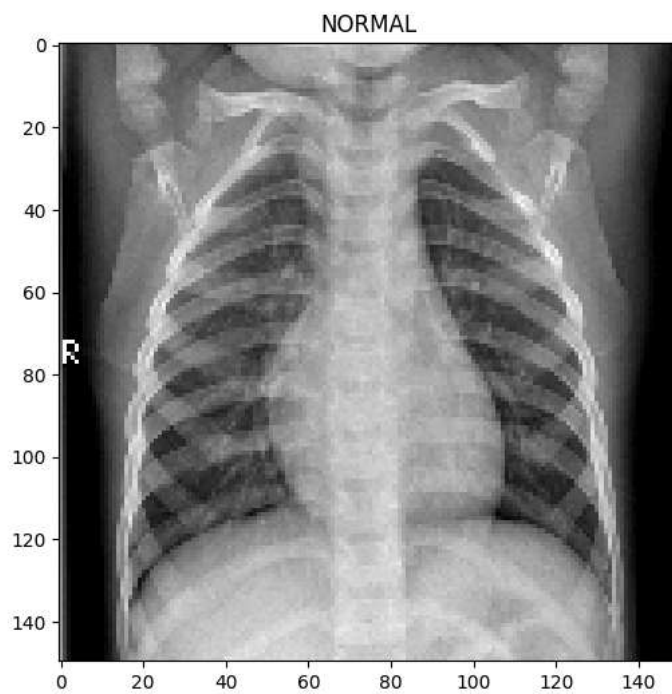
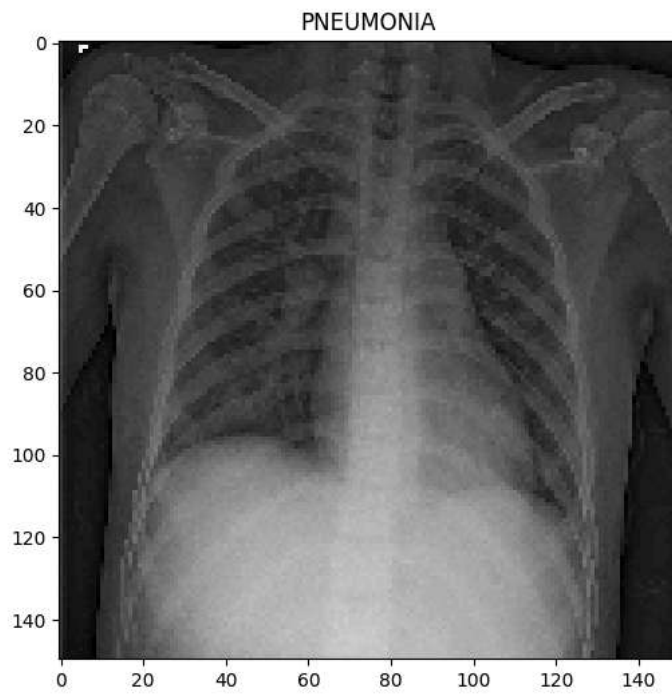
<Axes: xlabel='count'>



```
plt.figure(figsize=(6,6))
plt.imshow(train[0][0], cmap = 'gray')
plt.title(labels[train[0][1]])
```

```
plt.figure(figsize=(6,6))
plt.imshow(train[-1][0], cmap = 'gray')#last image
plt.title(labels[train[-1][1]])
#to display different images numbers can be changed
```

```
Text(0.5, 1.0, 'NORMAL')
```



```
x_train = []
y_train = []

x_val = []
y_val = []

x_test = []
y_test = []

for feat, label in train:
    x_train.append(feat)
    y_train.append(label)

for feat, label in val:
    x_val.append(feat)
    y_val.append(label)

for feat, label in test:
```

```
x_test.append(feat)
y_test.append(label)
```

```
x_train
```

```
[ 0, 0, 0, ..., 0, 5, 0],
[ 0, 0, 0, ..., 0, 0, 0]], dtype=uint8),
array([[26, 29, 31, ..., 40, 34, 24],
       [27, 28, 31, ..., 40, 36, 22],
       [27, 28, 30, ..., 38, 34, 3],
       ...,
       [23, 20, 15, ..., 32, 33, 32],
       [22, 20, 16, ..., 32, 33, 33],
       [20, 21, 16, ..., 32, 33, 36]], dtype=uint8),
array([[ 20, 19, 20, ..., 13, 17, 21],
       [ 20, 21, 20, ..., 11, 14, 21],
       [ 21, 19, 19, ..., 11, 13, 20],
       ...,
       [ 14, 16, 13, ..., 167, 171, 176],
       [ 13, 13, 15, ..., 171, 168, 176],
       [ 15, 14, 13, ..., 171, 172, 176]], dtype=uint8),
array([[ 48, 56, 59, ..., 65, 71, 56],
       [ 68, 66, 71, ..., 65, 71, 63],
       [ 82, 86, 86, ..., 68, 58, 63],
       ...,
       [ 13, 14, 16, ..., 1, 1, 2],
       [ 10, 9, 8, ..., 1, 1, 2],
       [ 24, 212, 9, ..., 1, 1, 1]], dtype=uint8),
array([[ 0, 0, 1, ..., 13, 8, 3],
       [ 0, 0, 1, ..., 13, 7, 3],
       [ 0, 0, 0, ..., 13, 7, 3],
       ...,
       [ 0, 0, 0, ..., 0, 0, 0],
       [ 0, 0, 0, ..., 0, 0, 0],
       [ 0, 0, 0, ..., 0, 0, 0]], dtype=uint8),
array([[151, 135, 116, ..., 134, 135, 135],
       [142, 132, 114, ..., 139, 137, 137],
       [141, 137, 108, ..., 134, 138, 135],
       ...,
       [160, 166, 167, ..., 88, 70, 58],
       [163, 171, 166, ..., 82, 69, 59],
       [164, 165, 167, ..., 83, 72, 57]], dtype=uint8),
array([[12, 15, 19, ..., 23, 19, 20],
       [10, 16, 19, ..., 28, 37, 24],
       [10, 13, 20, ..., 28, 25, 18],
       ...,
       [ 0, 0, 0, ..., 1, 1, 1],
       [ 2, 0, 0, ..., 1, 1, 2],
       [ 1, 0, 0, ..., 1, 1, 2]], dtype=uint8),
array([[66, 61, 66, ..., 53, 49, 47],
       [69, 68, 64, ..., 51, 51, 49],
       [85, 78, 70, ..., 53, 53, 49],
       ...,
       [32, 47, 53, ..., 22, 22, 22],
       [34, 49, 53, ..., 22, 22, 22],
       [34, 50, 58, ..., 22, 22, 22]], dtype=uint8),
array([[26, 40, 43, ..., 10, 12, 11],
       [36, 41, 44, ..., 31, 5, 9],
       [38, 46, 52, ..., 31, 30, 3],
       ...,
       [35, 35, 35, ..., 40, 41, 42],
       [35, 35, 35, ..., 41, 41, 42],
       [35, 35, 35, ..., 40, 42, 43]], dtype=uint8),
```

```
#normalize the data
x_train = np.array(x_train)/255
x_val = np.array(x_val)/255
x_test = np.array(x_test)/255
```

```
#grayscale single channel
#CNN in keras is expected expected channel dimension
x_train = x_train.reshape(-1, img_resize, img_resize, 1)
y_train = np.array(y_train)

x_val = x_val.reshape(-1, img_resize, img_resize, 1)
y_val = np.array(y_val)

x_test = x_test.reshape(-1, img_resize, img_resize, 1)
y_test = np.array(y_test)
```

```
#Data Augmentation(creating more data)
datagen = ImageDataGenerator(
    featurewise_center=False,#set input mean to zero
    featurewise_std_normalization=False,#dividing each input by its std
    rotation_range=30,#in degrees
    samplewise_center = False,#setting center of each sample as zero
    samplewise_std_normalization = False,
    zca_whitening = False,
    width_shift_range = 0.1,
    height_shift_range = 0.1,
    horizontal_flip= True,
    vertical_flip= False,
    zoom_range = 0.2
)

datagen.fit(x_train)
```

```
#Training the model
model= Sequential()
model.add(Conv2D(32,(3,3),strides = 1,padding = 'same', activation = 'relu', input_shape = (150,150,1)))
model.add(BatchNormalization())
model.add(MaxPool2D((2,2), strides = 2, padding = 'same'))

model.add(Conv2D(64,(3,3),strides = 1,padding = 'same', activation = 'relu'))
model.add(Dropout(0.1))# randomly deactivate 10% of the neuron from overfitting
model.add(BatchNormalization())
model.add(MaxPool2D((2,2),strides = 2, padding = 'same'))

model.add(Conv2D(64,(3,3),strides = 1,padding = 'same', activation = 'relu'))
model.add(BatchNormalization())
model.add(MaxPool2D((2,2),strides = 2, padding = 'same'))

model.add(Conv2D(128,(3,3),strides = 1,padding = 'same', activation = 'relu'))
model.add(Dropout(0.2))# randomly deactivate 20% of the neuron from overfitting
model.add(BatchNormalization())
model.add(MaxPool2D((2,2),strides = 2, padding = 'same'))

model.add(Conv2D(256,(3,3),strides = 1,padding = 'same', activation = 'relu'))
model.add(Dropout(0.2))# randomly deactivate 20% of the neuron from overfitting
model.add(BatchNormalization())
model.add(MaxPool2D((2,2),strides = 2, padding = 'same'))

model.add(Flatten())
model.add(Dense(units=128, activation = 'relu'))
model.add(Dropout(0.2))
model.add(Dense(units=1, activation = 'sigmoid'))

model.compile(optimizer = 'adam', loss = 'binary_crossentropy', metrics = ['accuracy'])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 150, 150, 32)	320
batch_normalization (BatchNormalization)	(None, 150, 150, 32)	128
max_pooling2d (MaxPooling2D)	(None, 75, 75, 32)	0
conv2d_1 (Conv2D)	(None, 75, 75, 64)	18,496
dropout (Dropout)	(None, 75, 75, 64)	0
batch_normalization_1 (BatchNormalization)	(None, 75, 75, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 38, 38, 64)	0
conv2d_2 (Conv2D)	(None, 38, 38, 64)	36,928
batch_normalization_2 (BatchNormalization)	(None, 38, 38, 64)	256
max_pooling2d_2 (MaxPooling2D)	(None, 19, 19, 64)	0
conv2d_3 (Conv2D)	(None, 19, 19, 128)	73,856
dropout_1 (Dropout)	(None, 19, 19, 128)	0
batch_normalization_3 (BatchNormalization)	(None, 19, 19, 128)	512
max_pooling2d_3 (MaxPooling2D)	(None, 10, 10, 128)	0
conv2d_4 (Conv2D)	(None, 10, 10, 256)	295,168
dropout_2 (Dropout)	(None, 10, 10, 256)	0
batch_normalization_4 (BatchNormalization)	(None, 10, 10, 256)	1,024
max_pooling2d_4 (MaxPooling2D)	(None, 5, 5, 256)	0
flatten (Flatten)	(None, 6400)	0
dense (Dense)	(None, 128)	819,328
dropout_3 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 1,246,401 (4.75 MB)

Trainable params: 1,245,313 (4.75 MB)

Non-trainable params: 1,088 (4.25 KB)

```
#commonly used during model training in libraries keras and tensorflow. job is to reduce the learning rate automatically if the r
learning_rate_reduction = ReduceLRonPlateau(monitor='val_accuracy',patience=2,verbose=1,factor=0.3, min_lr=0.000001)
#patience meaning if the validation accuracy does not improve for 2 consecutive epocs then it'll start reducing lr
#verbose it'll print the messages when the lr is reduced ... will let us know
#factor lr will be multiplied by 0.3(70% smaller)
#lr will not go beyond the min_lr
```

```
history=model.fit(x_train,y_train,batch_size=32,epochs=18,
                  validation_data=datagen.flow(x_val,y_val),
                  callbacks=[learning_rate_reduction])
```

```
Epoch 1/18
68/68 ————— 33s 271ms/step - accuracy: 0.9077 - loss: 0.3967 - val_accuracy: 0.7426 - val_loss: 5.0286 - learning_r
Epoch 2/18
68/68 ————— 4s 55ms/step - accuracy: 0.9520 - loss: 0.1274 - val_accuracy: 0.7426 - val_loss: 7.4946 - learning_r
Epoch 3/18
67/68 ————— 0s 29ms/step - accuracy: 0.9714 - loss: 0.0767
Epoch 3: ReduceLRonPlateau reducing learning rate to 0.0003000000142492354.
68/68 ————— 7s 105ms/step - accuracy: 0.9720 - loss: 0.0747 - val_accuracy: 0.7426 - val_loss: 5.1694 - learning_r
Epoch 4/18
68/68 ————— 4s 55ms/step - accuracy: 0.9841 - loss: 0.0415 - val_accuracy: 0.7426 - val_loss: 4.6091 - learning_r
Epoch 5/18
67/68 ————— 0s 28ms/step - accuracy: 0.9844 - loss: 0.0426
Epoch 5: ReduceLRonPlateau reducing learning rate to 9.000000427477062e-05.
68/68 ————— 4s 54ms/step - accuracy: 0.9865 - loss: 0.0359 - val_accuracy: 0.7426 - val_loss: 5.0677 - learning_r
```

```

Epoch 6/18
68/68 ————— 5s 75ms/step - accuracy: 0.9930 - loss: 0.0266 - val_accuracy: 0.7426 - val_loss: 4.1466 - learning_r
Epoch 7/18
67/68 ————— 0s 30ms/step - accuracy: 0.9965 - loss: 0.0163
Epoch 7: ReduceLROnPlateau reducing learning rate to 2.700000040931627e-05.
68/68 ————— 4s 55ms/step - accuracy: 0.9944 - loss: 0.0178 - val_accuracy: 0.7426 - val_loss: 3.7100 - learning_r
Epoch 8/18
68/68 ————— 4s 54ms/step - accuracy: 0.9972 - loss: 0.0138 - val_accuracy: 0.7426 - val_loss: 2.7976 - learning_r
Epoch 9/18
68/68 ————— 4s 60ms/step - accuracy: 0.9967 - loss: 0.0149 - val_accuracy: 0.7474 - val_loss: 2.1581 - learning_r
Epoch 10/18
68/68 ————— 4s 65ms/step - accuracy: 0.9967 - loss: 0.0122 - val_accuracy: 0.7694 - val_loss: 1.5177 - learning_r
Epoch 11/18
68/68 ————— 4s 55ms/step - accuracy: 0.9991 - loss: 0.0090 - val_accuracy: 0.7866 - val_loss: 1.2269 - learning_r
Epoch 12/18
68/68 ————— 4s 55ms/step - accuracy: 0.9967 - loss: 0.0143 - val_accuracy: 0.8086 - val_loss: 1.0408 - learning_r
Epoch 13/18
68/68 ————— 5s 76ms/step - accuracy: 0.9981 - loss: 0.0100 - val_accuracy: 0.8239 - val_loss: 0.8542 - learning_r
Epoch 14/18
68/68 ————— 4s 57ms/step - accuracy: 0.9963 - loss: 0.0127 - val_accuracy: 0.8278 - val_loss: 0.7662 - learning_r
Epoch 15/18
68/68 ————— 4s 55ms/step - accuracy: 0.9981 - loss: 0.0082 - val_accuracy: 0.8182 - val_loss: 0.9135 - learning_r
Epoch 16/18
67/68 ————— 0s 29ms/step - accuracy: 0.9964 - loss: 0.0130
Epoch 16: ReduceLROnPlateau reducing learning rate to 8.100000013655517e-06.
68/68 ————— 4s 64ms/step - accuracy: 0.9977 - loss: 0.0116 - val_accuracy: 0.8182 - val_loss: 0.8333 - learning_r
Epoch 17/18
68/68 ————— 5s 75ms/step - accuracy: 0.9981 - loss: 0.0087 - val_accuracy: 0.8182 - val_loss: 0.8303 - learning_r
Epoch 18/18
68/68 ————— 4s 55ms/step - accuracy: 0.9967 - loss: 0.0100 - val_accuracy: 0.8373 - val_loss: 0.7635 - learning_r

```

```
model.evaluate(x_test,y_test)
```

```

20/20 ————— 1s 60ms/step - accuracy: 0.7260 - loss: 1.8520
[1.8520092964172363, 0.7259615659713745]

```

```

print(f"loss of the model is {model.evaluate(x_test,y_test)[0]}")
print(f"Accuracy of the model is {model.evaluate(x_test,y_test)[1]*100}%")

```

```

20/20 ————— 0s 7ms/step - accuracy: 0.7260 - loss: 1.8520
loss of the model is 1.8520092964172363
20/20 ————— 0s 8ms/step - accuracy: 0.7260 - loss: 1.8520
Accuracy of the model is 72.59615659713745%

```

```

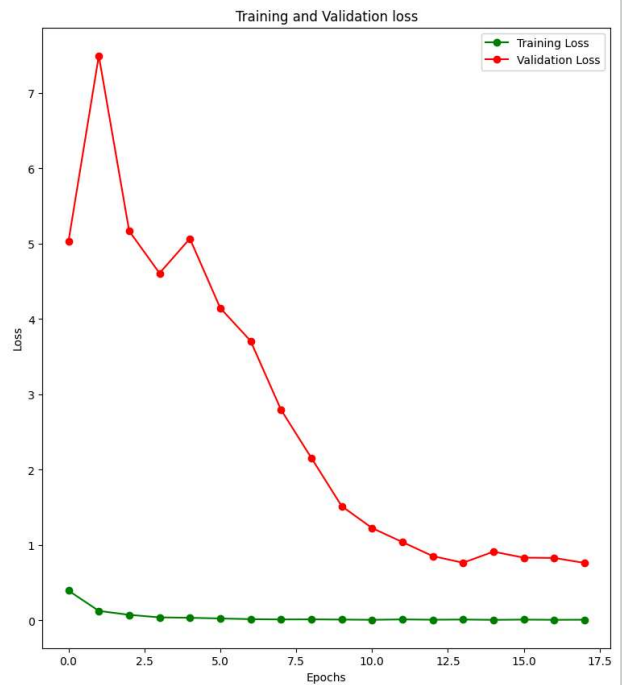
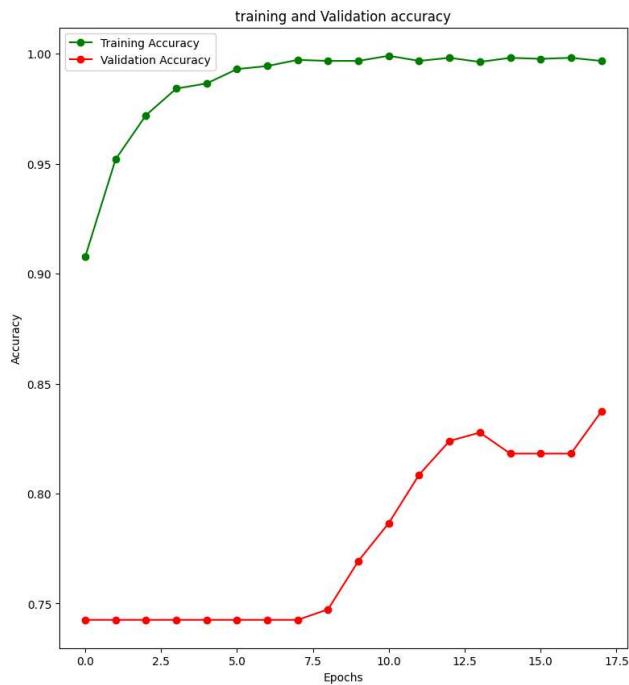
epochs = [i for i in range(18)]
fig, ax = plt.subplots(1,2)
train_acc = history.history['accuracy']
train_loss = history.history['loss']
val_acc = history.history['val_accuracy']
val_loss = history.history['val_loss']

fig.set_size_inches(20,10)

ax[0].plot(epochs, train_acc, 'go-', label = 'Training Accuracy')
ax[0].plot(epochs, val_acc, 'ro-', label = 'Validation Accuracy')
ax[0].set_title('training and Validation accuracy')
ax[0].legend()
ax[0].set_xlabel("Epochs")
ax[0].set_ylabel("Accuracy")

ax[1].plot(epochs, train_loss, 'g-o', label = 'Training Loss')
ax[1].plot(epochs, val_loss, 'r-o', label = 'Validation Loss')
ax[1].set_title('Training and Validation loss')
ax[1].legend()
ax[1].set_xlabel("Epochs")
ax[1].set_ylabel("Loss")
plt.show()

```



```
predictions = (model.predict(x_test)>0.5).astype('int32')
#0 for Pneumonia
# 1 for normal chest x-ray
pedictions = predictions.reshape(1,-1)[0]
predictions[:15] #first 15
```

20/20 ————— 1s 41ms/step

```
array([[0],
       [0],
       [0],
       [0],
       [0],
       [0],
       [0],
       [0],
       [0],
       [0],
       [0],
       [0],
       [0],
       [0],
       [0]], dtype=int32)
```

```
print(classification_report(y_test, predictions, target_names = ['Pneumonia(Class 0)','Normal(class 1)']))
```

	precision	recall	f1-score	support
Pneumonia(Class 0)	0.70	1.00	0.82	390
Normal(class 1)	0.98	0.27	0.43	234
accuracy			0.73	624
macro avg	0.84	0.64	0.62	624
weighted avg	0.80	0.73	0.67	624

```
cm = confusion_matrix(y_test,predictions)
cm
```

```
array([[389,  1],
       [170, 64]])
```

```
correct = np.nonzero(predictions == y_test)[0]
incorrect = np.nonzero(predictions != y_test)[0]
#
```



```
#some correctly predicted classes

i=0
plt.figure(figsize=(6,15))
for c in correct[10:18]:
    plt.subplot(4,2,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(x_test[c].reshape(150,150), cmap = 'gray', interpolation= 'none')
    plt.title("Predicted Class {},Actual Class {}".format(predictions[c],y_test[c]))
    plt.tight_layout()
    i += 1
```

Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



```
#some incorrectly predicted classes
```

```
i=0
```

```
plt.figure(figsize=(6,15))
for c in incorrect[10:18]:
    plt.subplot(4,2,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(x_test[c].reshape(150,150), cmap = 'gray', interpolation= 'none')
    plt.title("Predicted Class {},Actual Class {}".format(predictions[c],y_test[c]))
    plt.tight_layout()
    i += 1
```

Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



Predicted Class [0],Actual Class 0



```
#now by using rmsprop optimizr  
model= Sequential()  
model.add(Conv2D(32,(3,3),strides = 1,padding = 'same', activation = 'relu', input_shape = (150,150,1)))  
model.add(BatchNormalization())
```

```
model.add(MaxPool2D((2,2), strides = 2, padding = 'same'))

model.add(Conv2D(64,(3,3),strides = 1,padding = 'same', activation = 'relu'))
model.add(Dropout(0.1))# randomly deactivate 10% of the neuron from overfitting
model.add(BatchNormalization())
model.add(MaxPool2D((2,2),strides = 2, padding = 'same'))

model.add(Conv2D(64,(3,3),strides = 1,padding = 'same', activation = 'relu'))
model.add(BatchNormalization())
model.add(MaxPool2D((2,2),strides = 2, padding = 'same'))

model.add(Conv2D(128,(3,3),strides = 1,padding = 'same', activation = 'relu'))
model.add(Dropout(0.2))# randomly deactivate 20% of the neuron from overfitting
model.add(BatchNormalization())
model.add(MaxPool2D((2,2),strides = 2, padding = 'same'))

model.add(Conv2D(256,(3,3),strides = 1,padding = 'same', activation = 'relu'))
model.add(Dropout(0.2))# randomly deactivate 20% of the neuron from overfitting
model.add(BatchNormalization())
model.add(MaxPool2D((2,2),strides = 2, padding = 'same'))

model.add(Flatten())
model.add(Dense(units=128, activation = 'relu'))
model.add(Dropout(0.2))
model.add(Dense(units=1, activation = 'sigmoid'))

model.compile(optimizer = 'RMSProp', loss = 'binary_crossentropy', metrics = ['accuracy'])

model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_5 (Conv2D)	(None, 150, 150, 32)	320
batch_normalization_5 (BatchNormalization)	(None, 150, 150, 32)	128
max_pooling2d_5 (MaxPooling2D)	(None, 75, 75, 32)	0
conv2d_6 (Conv2D)	(None, 75, 75, 64)	18,496
dropout_4 (Dropout)	(None, 75, 75, 64)	0
batch_normalization_6 (BatchNormalization)	(None, 75, 75, 64)	256
max_pooling2d_6 (MaxPooling2D)	(None, 38, 38, 64)	0
conv2d_7 (Conv2D)	(None, 38, 38, 64)	36,928
batch_normalization_7 (BatchNormalization)	(None, 38, 38, 64)	256
max_pooling2d_7 (MaxPooling2D)	(None, 19, 19, 64)	0
conv2d_8 (Conv2D)	(None, 10, 10, 128)	73,856